

PA 1: Implementing xv6 User Commands and System Calls

Repository Link: <https://github.com/miguel-desantiago/xv6-riscv/tree/pa1-syscalls>

Task 1. (5 pts) Implement the uptime utility

My uptime.c code is found in the user directory in the pa1 branch of my xv6 repo. The following screenshot shows the results from running my uptime command...

```
miguel-de-santiago@miguel-de-santiago-VMware-Virtual-Platform:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ uptime
up 93 clock ticks
$
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

My time1.c code is found in the user directory in the pa1 branch of my xv6 repo. The following screenshot shows the results from running the time1 command with various examples.

```
miguel-de-santiago@miguel-de-santiago-VMware-Virtual-Platform:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ time1 matmul
Time: 14 ticks
elapsed time: 14 ticks
$ time1 matmul & time1 matmul &
$ Time: 25 ticks
elapsed time: 26 ticks
Time: 25 ticks
elapsed time: 26 ticks
time1 sleep 10
elapsed time: 10 ticks
$
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

Per the instructions, we were first tasked with creating a new cputime field to a struct called

```
found:
p->pid = allocpid();
p->state = USED;

// Set initial cputime to 0
p->cputime = 0;
```

'proc' in kernel/proc.h. To do this, I went into the file and identified the proc struct. In that struct, I added a new field, "uint64 cputime." The purpose of this change was to add the cputime field/variable that was going to be in charge of tracking the cputime a process has used (number of times it uses up its time slices).

```
char name[16]; // Process name (debugging)

// Tracking time when process uses up its timeslice
uint64 cputime;

};
```

After making that change, I needed to initialize

the field to zero whenever a process was created. In order to do this, I had to edit the file "kernel/proc.c" to find the allocproc() method and make the change. Within this method, I identified the lines where the PID was assigned to the process and the state was updated. This reflected when the process was actually created. After those two lines, I added a new line, "p->cputime=0" to set the initial cputime for that process created to 0.

Next, I needed to work on updating this field whenever a process used up its time slice. There were two scenarios that I needed to handle, the usertrap() and the kerneltrap(). The user trap focused on handling traps that occurred when running user programs, while the kernel trap handled traps occurring when running kernel code. To handle these changes, I first went into kernel/trap.c and identified the usertrap() method.

In the usertrap() method, I found the line "if (which_dev == 2) {yield()}." This line indicated that the trap that had been identified was from a timer interrupt, and thus had to give up the CPU.

```
// give up the CPU if this is a timer interrupt.
if (which_dev == 2){
    // Increase cputime field, time slice has been used up
    p->cputime++;
    yield();
}
```

This line meant that the process had used up its time slice, which meant I had to update the cputime field for that process. I added a new line "p->cputime++" before the yield(), since it was the point in which the time slice had run out.

After this, I followed similar logic in kerneltrap() and identified the code that handled timer interrupts. However, since kerneltrap() can handle kernel code traps that may not necessarily be associated with a process (shown by myproc() != 0), I had to first gather the information of the process with myproc(), store it in a variable, and then update that process' cputime field.

```
// give up the CPU if this is a timer interrupt.
if (which_dev == 2 && myproc() != 0){
    // Gather the process information and update cputime field
    struct proc *p = myproc();
    p->cputime++;
    yield();
}
```

After these steps, I had completed the task.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

All code can be found on the pa1-syscalls branch of my xv6 forked repository.

kernel/

pstat.h

This was a new file created as part of the task instructions. Its purpose was to define the struct 'rusage', which would be used later in the task to store and return rusage for a child process.

syscall.h

The only change made to this file was adding a new entry for the wait2() method, with a new associated system call number. The purpose of this was establishing it as one of the system calls that the kernel will now need to manage.

syscall.c

The only changes made to this file was editing the array that maps the syscall numbers from syscall.h to the actual functions that handle them, along with adding sys_wait2 to the prototypes. All that was needed was mapping SYS_wait2, which was added in syscall.h, and mapping it to the function sys_wait2.

sysproc.c

This file was one of the primary changes made with the addition of the sys_wait2() function. The purpose of adding this function was to begin the handling of our wait2() function that will provide a process' status and its cputime. The function sys_wait2() in this file was almost a direct copy of the sys_wait() function, with a simple addition of a rusage_address variable, that was passed into argaddr() in a similar way as the sys_wait() function had done it. From there, the function called kwait2() with our two addresses to begin gathering the information that would be placed in these addresses.

proc.c

This file was another primary change that was done with the addition of the kwait2() function. The purpose of this was to create the logic, similar to kwait(), that would search for exited child processes and provide the status and rusage information. The code itself was very similar to that of kwait(), however, it required an additional copyout() call to be able to save the additional rusage information. One small additional change made to this file was including "pstat.h," which is the file where we defined the rusage struct, which will be utilized in our kwait2() function as a way to store the rusage information of a process (cputime).

defs.h

This file was a very minor change that only involved adding the kwait2() function declaration. This line served the purpose of making kwait2() visible outside of proc.c so that it is callable by other places.

user/

user.h

The first change to this file was adding the line 'struct rusage;' to the file. This was done to ensure we could use the previously defined struct 'rusage' in user-process code.

The second change to this file was adding the method declaration for wait2() with the purpose of providing a potential system call users could make.

usys.pl

The only change made to this file was adding a new entry for the wait2() method we will be creating. The purpose of adding it to this file is to continue establishing wait2() as a potential system call that users can make.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

My time.c code can be found in the user directory in the pa1-syscalls branch of my xv6 forked repository.

```
miguel-de-santiago@miguel-de-santiago-VMware-Virtual-Platform:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,i
d=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ matmul
Time: 10 ticks
$ time matmul
Time: 17 ticks
elapsed time: 17 ticks, cputime: 17 ticks, 100% CPU
$ time matmul &
Time: 19 ticks
elapsed time: 20 ticks, cputime: 10 ticks, 50% CPU
Time: 19 ticks
elapsed time: 20 ticks, cputime: 11 ticks, 55% CPU
time sleep 10
elapsed time: 10 ticks, cputime: 0 ticks, 0% CPU
$
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This programming assignment covered a wide range of topics through a series of activities that built on each other. Since PA1 was the first programming assignment we have had, that aspect definitely added to its difficulty level, working in a new VM environment with unfamiliar tools, code, files, etc. In my opinion, the most difficult aspect of PA1 was understanding the concepts we were utilizing and trying to use that to find solutions to the problems. Writing the code itself was not very difficult, as most code was not very complex. Rather, the complexity of the code came from figuring out the logic that had to go behind it. For instance, something as simple as a fork required a bit more low-level knowledge and understanding for this assignment than just a general overview.

One of the major difficulties I faced was with Task #4. Conceptually, this task was the most difficult as it required a lot of understanding of how the kernel communicated with user programs and wrote data into a user's processes. This task was easily the task I spent the most

time on, and required a lot of review to understand how exactly all of the components were communicating with each other to serve the final purpose. Once I was able to gather this understanding, the implementation made a lot more sense, and I was able to conceptually understand why the code needed to be written in a certain way.

In conclusion, this assignment was definitely very useful for understanding the concepts we have covered thus far in the course and it served as a great way to gain a more low-level understanding. Moving forward, I think continuing to treat these assignments as review guides and study sessions is incredibly helpful, as it helps me learn so much more as I go and be able to put my knowledge into practice with the code I write.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

One potential limitation with the time command comes from forking child processes and, thus, creating grandchild processes. The time command that is currently implemented tracks the cputime usage and elapsed time of the child processes of that command. However, let's say a scenario exists where those child processes being tracked then forked to create new child processes, and now grandchild processes exist for the original time command. Those grandchild processes are not being tracked as part of the cputime usage and elapsed time. They have their own fields, but time is only ever looking at its children. Thus, you can have grandchild processes utilize the CPU and they would not be accounted into the cputime that is being calculated. Thus, this would lead to a limitation where forking child processes could cause a situation where the true cputime was not represented.

AI Use Acknowledgement

I utilized Anthropic's Claude (Sonnet 5, High Effort) throughout this assignment, predominantly as a study partner, tutor, and pair-programming partner. Claude did not write code for me; rather, it helped me review course concepts covered in this assignment (forks, traps, waits, etc.) and assisted in reviewing code I had already written to identify bugs and verify logic, always without providing the answer directly but rather guiding towards the answers with questions, requiring me to reason and defend fixes myself.